

DSAII- HW1

Tomas Salaz

January 30, 2026

Problem 1

Prove $\log n! = \Theta(n \log n)$ and that $n! = \omega(2^n)$ and $n! = o(n^n)$

Part 1: Prove $\log n! = \Theta(n \log n)$

Will be done with a Direct Proof showing the Upper and Lower Bounds are the same, which is analogous to Θ

$$\log n! = \log(1 \cdot 2 \cdot 3 \cdot \dots \cdot n) = \log 1 + \log 2 + \log 3 + \dots + \log n$$

For each term, $\log k \leq \log n$ for $k \leq n$.

Thus, $\log n! \leq n \cdot \log n$

This defines the lower bound which can be expressed as $\log n! = \Omega(n \cdot \log n)$

Now we must show that the upper bound O , is the same as the lower bound.

Consider the 2nd half of the term from $n/2 \dots n$.

$$\log n! = \log 1 + \log 2 + \dots + \log n$$

$$\log n \geq \log (n/2) + \log (n/2) + 1 + \dots + \log n$$

Each of these $n/2$ terms is at least $\log (n/2)$, Thus

$$\log n! \geq (n/2) \cdot \log (n/2)$$

$$\log n! \geq (n/2) \cdot (\log n - \log 2)$$

$$\log n! \geq (n/2) \log n - (n/2) \log 2$$

As $n \rightarrow \infty$, we can ignore the constant term $\log 2$, which leaves $O(n \log n)$ for the upper bound.

Since the Upper Bound and the Lower Bound have been proven to be the same, we directly prove that $\log n! = \Theta(n \log n)$

Part 2: Prove $n! = \omega(2^n)$

ω implies that we need to show $\lim(n \rightarrow \infty) n!/2^n = \infty$

$$n!/2^n = (1 \cdot 2 \cdot 3 \cdot \dots \cdot n)/2^n = (1/2) \cdot (2/2) \cdot (3/2) \cdot \dots \cdot (n/2)$$

Now we only consider terms $n \geq 3$ since we need to show the $\lim n \rightarrow \infty$, thus the product of the terms from 3 to n is

$$(3/2) \cdot (4/2) \cdot \dots \cdot (n/2) = (3 \cdot 4 \cdot \dots \cdot n)/(2^{n-2})$$

Thus, $n!/2^n$ is approximates the product, and $\lim (n \rightarrow \infty)(n!/2^n) \rightarrow \infty$
 Therefore, $n! = \omega(2^n)$

Part 3: Prove $n! = o(n^n)$

o notation implies that we need to show $\lim(n \rightarrow \infty) n!/n^n = 0$, by Direct Proof

$$n!/n^n = (1 \cdot 2 \cdot 3 \cdot \dots \cdot n)/n^n = (1/n) \cdot (2/n) \cdot (3/n) \cdot \dots \cdot (n/n)$$

Each term $k/n \leq 1$ thus all terms other than the last are ≤ 1 .

Therefore $n!/n^n = (1/n) \cdot (2/n) \cdot \dots \cdot ((n-1)/n) \cdot 1 \leq 1/n$

Since $1/n \rightarrow 0$ as $n \rightarrow \infty$, we directly prove that $n! = o(n^n)$

Problem 2

Part A: $\log_2 f(n)$ is $O(\log_2 g(n))$

FALSE, Proof by Counter Example

Let $f(n) = 2$ and $g(n) = 1$.

By definition of O there must exist c such that $f(n) \leq c \cdot g(n)$, which is true is $c = 1$

Now we must show that $\log_2 f(n)$ is $O(\log_2(g(n)))$

$$\log_2(2) = O(\log_2(1))$$

$$\log_2(2) = 1$$

$$\log_2(1) = 0$$

Thus, by Definition of O , we must have $1 \leq c \cdot 0$ which is impossible, hence False.

Part B: $2^{f(n)}$ is $O(2^{g(n)})$

FALSE, Proof by Counter Example

Let $f(n) = 2n$ and $g(n) = n$

Same properties of O must hold, in this case $2n \leq c \cdot n$

Now we must show that $2^{f(n)}$ is $O(2^{g(n)})$

$$2^{f(n)} = 2^{2n} = 4^n$$

$$2^{g(n)} = 2^n$$

Based on the O definition, $4^n \leq c \cdot 2^n$

$$4^n \leq c \cdot 2^n$$

$$4^n/2^n \leq c$$

This means that $2^n \leq c$. Since $n \rightarrow \infty$, there is no constant that can satisfy this condition, hence False.

Part C: $f(n)^2$ is $O(g(n)^2)$

TRUE, Direct Proof.

We know that O by definition says $f(n) \leq c \cdot g(n)$ for all $n \geq n_0$

$$f(n) \leq c \cdot g(n)$$

$$[f(n)]^2 \leq [c \cdot g(n)]^2$$

$$[f(n)]^2 \leq [c]^2 \cdot [g(n)]^2$$

This is exactly the definition of O given that $c = c^2$ thus exists, hence True.

Problem 3

Recurrence Derivation

Since $n = 9$, there are 10^n possible total strings to consider, since each position can contain any digit from 1 to 10.

Let $f(n)$ be the number of strings of length n with at least one 9.

Let $g(n)$ be the number of strings of length n with no 9's.

With this we can make some assumptions

$$(1) f(n) = 10^n - g(n)$$

$$(2) f(n) + g(n) = 10^n$$

For strings without a 9, any of the digits have 9 possibilities (0-8). Thus

$$g(n) = 9 \cdot g(n-1)$$

We can use this term in assumption (1) from above.

$$f(n) = 10^n - [9 \cdot g(n-1)]$$

We know that $g(n-1) = 10^{n-1} - f(n-1)$ by substituting in $(n-1)$ and rearranging terms.

$$g(n) = 10^n - f(n)$$

$$g(n-1) = 10^{n-1} - f(n-1)$$

We can use the term from the line above to continue.

$$f(n) = 10^n - [9 \cdot g(n-1)]$$

$$f(n) = 10^n - 9 \cdot [10^{n-1} - f(n-1)]$$

$$f(n) = 10^n - 9 \cdot 10^{n-1} + 9 \cdot f(n-1)$$

$$f(n) = 10 \cdot 10^{n-1} - 9 \cdot 10^{n-1} + 9 \cdot f(n-1)$$

$$f(n) = 10^{n-1} + 9 \cdot f(n-1)$$

Thus the recurrence relation is $f(n) = 9 \cdot f(n-1) + 10^{n-1}$

Solving the Recurrence

For $n = 1$ there is only one possible string that contains the digit 9, thus $f(1) = 1$

For $n = 2$ we can use the recurrence relation, $f(2) = 9 \cdot f(1) + 10^1 = 9 + 10 = 19$ which is true since there are 100 total strings, and we can assume there are only 9 choices for digits without a 9, and 2 spots, thus $100 - (9 \cdot 9) = 19$

We can solve the recurrence using guess and check, but having more terms would be useful.

$$f(3) = 9 \cdot 19 + 10^2 = 171 + 100 = 271$$

$$f(4) = 9 \cdot 271 + 10^3 = 2439 + 1000 = 3439$$

Based on the check from the $f(2)$ case, I tried using the same form, $f(n) = 10^n - 9^n$, for $f(3)$ and $f(4)$, so we can use this to guess and check.

For case $f(1)$ we get $f(1) = 10 - 9 = 1$ so this holds.

For the recurrence relation we need to prove $10^n - 9^n = 9 \cdot f(n-1) + 10^{n-1}$

$$\begin{aligned}
& 9 \cdot f(n-1) + 10^{n-1} \\
& 9 \cdot (10^{n-1} - 9^{n-1}) + 10^{n-1} \\
& 9 \cdot 10^{n-1} - 9 \cdot 9^{n-1} + 10^{n-1} \\
& 10 \cdot 10^{n-1} - 9^n \\
& 10^n - 9^n \\
& \text{Therefor, } f(n) = 10^n - 9^n
\end{aligned}$$

Θ Bounds

$f(n) = 10^n - 9^n = 10^n(1 - (9/10)^n)$

Upper Bound: $f(n) = 10^n - 9^n < 10^n$ so $f(n)$ is $O(10^n)$

Lower Bound: $10^n - 10^n \cdot (9/10)^n = 10^n(1 - (9/10)^n) \geq 10^n \cdot (1 - (9/10)) = 10^n(1/10)$, thus $\Omega(10^n)$

Since $O(10^n) = \Omega(10^n)$ then $\Theta(10^n)$ is true.

Problem 4

We want to Prove that the number of leaves in a balanced binary tree is always 2^k for $k \geq 0$.

Proof by Induction on Number of Nodes, n.

BC

A tree with a single node contains only the root which is the leaf. $2^0 = 1$ thus the number of leaves is a power of 2, which is trivially true.

IH

For all Leaf Balanced Binary Trees with fewer than n nodes, the number of leaves is a power of 2.

IS

Case 1: The Tree T has one subtree called T_c

T_c contains all the nodes of T so T_c has n-1 nodes. T_c is also a leaf-balanced Tree and has the same properties. By the IH, since T_c has $n - 1$ nodes, which is less than T having n nodes, T_c must a number of leaf nodes that is a power of 2.

Case 2: The Tree T has 2 subtrees a Right and a Left called, T_R and T_L . The number of Nodes in the Tree is $N_{Total} = N_R + N_L + 1$. We can also assume that T_R and T_L are Leaf Balanced Trees and have the same properties.

When we apply the IH to both subtress, T_R and T_L , the assumption holds since the number of nodes in either subtree is less than the total nodes in the whole Tree. We can define the Leaf nodes in either subtree as $LN_R = 2^j$ and $LN_L = 2^k$.

The sum of the leaves in the subtrees is the total amount of leaves in the entire tree. Thus, $LN_{Total} = 2^j + 2^k$ which is a power of 2.